

anime.js v4.5.0 を実装で使う

2026.09.16

Hol1kgmg(ほりかわ)

自己紹介

Hol1kgmg(ほりかわ)

- WebアプリのFE・BEエンジニア
- キーボード、nixが好き
- anime.js の性能持て余しがち

 @Hol1kgmg

 Hol1kgmg

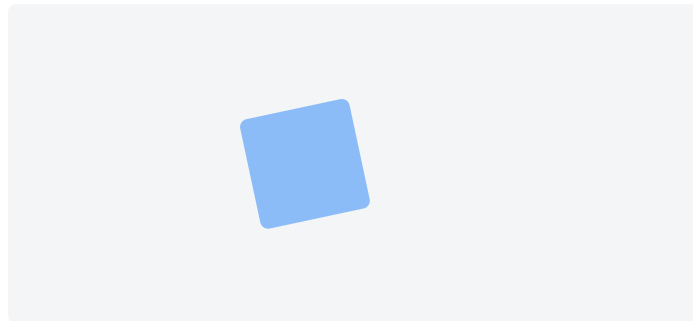


anime.js とは

DOM を時間で動かすライブラリ

- 対象は**DOM要素**・**SVG**・**ただのJSオブジェクト**
- 値を補間する仕組みと、**時計**・**イージング**・**再生制御**を持つ

```
animate('.box', {  
  x: 100,  
  rotate: 360,  
  duration: 1000,  
})
```



Three.js とは

3Dオブジェクトを画面に描くライブラリ

- 対象は Mesh / Camera / Light
- 描画は自前の requestAnimationFrame ループ。

値は**毎フレーム自分で更新する**

```
const tick = () => {  
  mesh.rotation.y += 0.01 // ラジアン  
  renderer.render(scene, camera)  
  requestAnimationFrame(tick)  
}
```

v4.5.0 で adapter が来た

adapter は「読み方・書き方」を教える仕組み

- これまで `animate()` が動かせたのは、`anime.js` が扱い方を知っているもの（DOM要素など）だけ
- v4.5.0 で追加された `registerAdapter()` は、その扱い方を自分で教えられるAPIになった
- 教えることは2つだけ。その値をどう読むかと、どう書き込むか

```
import { registerAdapter } from 'animejs/adapters'

// ① どのようなものを動かしたいか
const adapter = registerAdapter()
const speaker = adapter.registerTargetAdapter(t => t instanceof Speaker)

// ② その中の値の読み方・書き方
speaker.registerProperty('volume',
  s => s.volume, // 読む
  (s, value) => { s.volume = value }, // 書く
)

animate(speaker, { volume: 100, duration: 1000 }) // 動かせるようになる
```

three.js の分は公式が用意してくれている

前のページの登録作業を、three.js 向けに済ませたものが v4.5.0 に同梱された

```
import 'animejs/adapters/three' // 読み込むだけ。受け取る変数もない
import { animate } from 'animejs'

animate(mesh, { x: 3, rotateY: 360, opacity: 0.3, color: ['#0af', '#f0a'] })
```

- 自分で `registerAdapter()` を書く必要はなく、**import** を1行足すだけ
- 以降は `mesh` ・ マテリアル ・ ライト ・ カメラ ・ 音まで、そのまま `animate()` に渡せる
- 値を直接セットする `utils.set()` も同じように使えるようになる

Before → After

Before → After

```
1 import 'animejs/adapters/three'
2
3 animate(mesh, {
4   x: 100,
5   rotateY: 360,
6   duration: 1000,
7 })
```

ターゲットが `mesh` そのものになるので、**配列を渡す**・`stagger` **を使う**もそのまま効く

何ができるか

実例① Object3D を動かす

```
animate(mesh, {  
  x: [-3, 3], // [from, to]  
  rotateY: 360,  
  opacity: 0.3, // material に transparent: true が必要  
  duration: 1200,  
  ease: 'inOutSine',  
  loop: true,  
  alternate: true,  
})
```

実例② camera / light

```
animate(camera, { fov: [50, 30], duration: 1200 })  
  
animate(spotLight, { intensity: [30, 150], color: ['#0af', '#f0a'], duration: 1200 })
```

spotLight.intensity / color – 光量と色

anime.js で three.js を使うと

DOM と 3D が同じ timeline に乗る



裏側のコードの変化

```
1 // コメント (DOM)
2 tl.add(commentEl, { x: [w, -commentEl.offsetWidth], duration: 4200, ease: 'linear' }, at)
3
4 // リール (3D) - 等速フェーズ / 減速フェーズ
5 tl.add(group, { rotateX: [start, DECEL_DEG], duration: decelAt(k), ease: 'linear' }, 0)
6 tl.add(group, { rotateX: [DECEL_DEG, 0], duration: DECEL, ease: 'outQuint' }, decelAt(k))
```

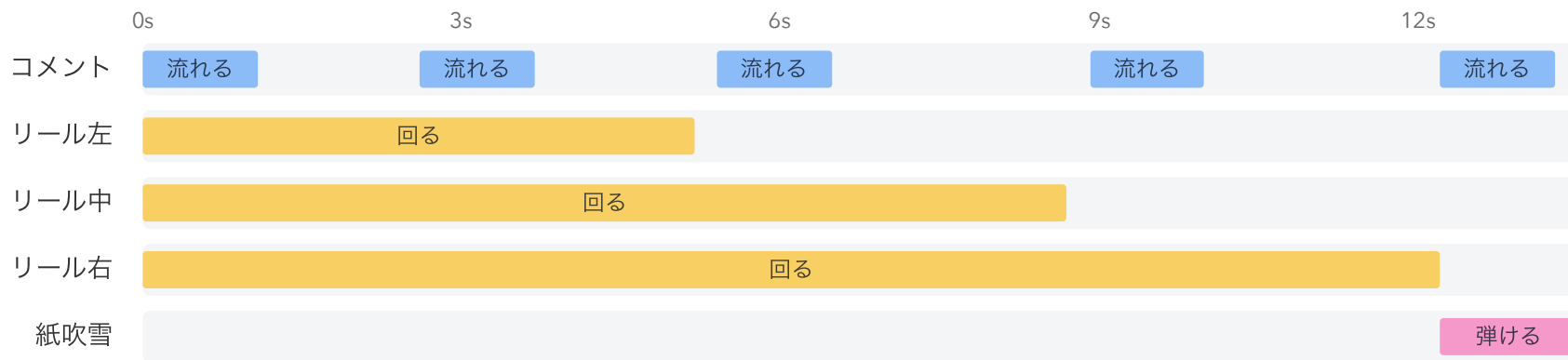
アニメーション表現の比較

Three.js のアニメーション



基準となるものがthree.jsにない。揃えるには JavaScript 側で時刻を計算する

anime.js のアニメーション



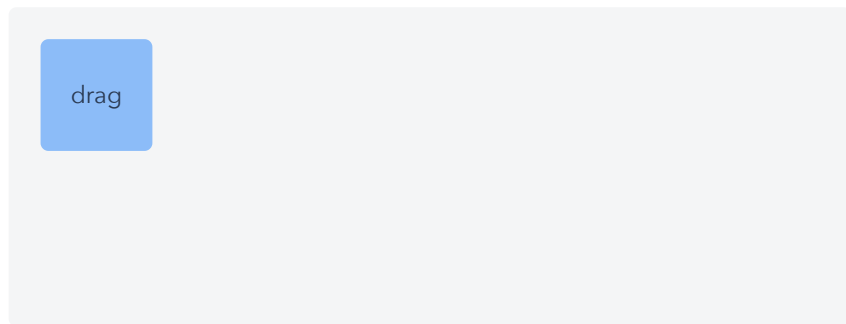
共通の時間軸が1つ。いつ何が起きるかが1つのスケジュール上で構築される

adapter にできないこと

adapter が拡張したのは「値の書き込み先」だけ

オブジェクトを動かす API (`animate` / `utils.set` / `createTimeline`) は使えるが、
入力を扱う API (`createDraggable` / `onScroll`) は **DOM 専用のまま**

`createDraggable()`



`onScroll()`



anime.js の面白い要素でもある「ユーザー操作と連動するアニメーション」は three.js adapter で対応していない

まとめ

役割分担

three.js は**3Dを描く**担当、
anime.js は**時間で動かす**担当

必要なのはimport 1行

3DのオブジェクトをDOMと同じ書き方で渡せる

変わったこと

バラバラに動いていたものを、1本の時間軸に並べて演出できる

限界

つながったのは**動かす側だけ**。
ドラッグやスクロールは、いままDOM専用